

User Defined Data Types

- User created datatype that can be customized
- good for when software does not provide a datatype
- reduces Code complexity

Data Types Non Composite: Can Hold ONE value, Does NOT depend on any other datatypes

- Built in (STRING, CHAR, INTEGER, REAL, BOOLEAN)
- User Defined (Enumerated, Pointer)

User Defined Non Composite: Enumerated

- A list that is ALWAYS in order/Ordered list of elements

```
TYPE Day
    DECLARE Today : (Mon,Tue,Wed,Thu,Fri,Sat,Sun)
ENDTYPE

DECLARE MyDay : Day
MyDay.Today <-- Tue
```

REMEMBER, string values are NOT enclosed in Commas here

Pointers

- Pointers point to memory locations

Memory Location	Values
101	Ayan
102	Hassan
103	Haji

Lets say pointer is at 101 and is called Name OUPUT Name will output Ayan OUPUT ^Name will output 101 OUTPUT Name^ will output Ayan

- When a pointer points toward a memory location (^Name)

- When a pointer points toward a value and not a memory location (Name[^])

```
TYPE MyPointer = ^INTEGER
```

- Makes a pointer that points to an integer

Composite: Holds multiple values, same or different

- Built in (Array, Linked List, Stack, Queue)
- User Defined ("Record", "CLASS", "SET")

User Defined Composite: Record

- Collection of different datatypes represented as one identifier

```
TYPE Student
  DECLARE Surname : STRING
  DECLARE Age : INTEGER
  DECLARE Mark : REAL
ENDTYPE

DECLARE MyStudent : Student
MyStudent.Surname <-- "Esa"
MyStudent.Age <--- 15
MyStudent.Mark <--- 90.5
```

CLASS

- We'll cover this later in classes(yes include this in the website)
- A collection of "attributes" and "methods" having different "relationships" applied to them
- Used in Object Oriented Programming(OOP)

SET

- Unordered List of elements
- Which we apply different Mathematical Operations E.g A,"A"